

Pose Estimation

Pose Estimation

- 1. Model Introduction
- 2. Pose Estimation: Image
 - Preview
- 3. Pose Estimation: Video
 - Preview
- 4. Pose Estimation: Real-time Detection
 - 4.1 USB Camera
 - Preview
- References

This section demonstrates **Ultralytics** Pose Estimation on images, videos, and real-time detection using Python.

1. Model Introduction

Pose estimation is a task that involves identifying the positions of specific points (usually called keypoints) in an image. Keypoints can represent various parts of an object, such as joints, landmarks, or other distinctive features. The positions of keypoints are typically represented as a set of 2D `[x, y]` or 3D `[x, y, visible]` coordinates.

The output of a pose estimation model is a set of points representing the keypoints of objects in an image, usually including a confidence score for each point. Pose estimation is an excellent choice when you need to identify specific parts of an object in a scene and understand their spatial relationships with each other.

In the default pose model of YOLO26, there are 17 keypoints, each representing a different body part. Here is the mapping from each index to the corresponding body joint:

1. Nose
2. Left Eye
3. Right Eye
4. Left Ear
5. Right Ear
6. Left Shoulder
7. Right Shoulder
8. Left Elbow
9. Right Elbow
10. Left Wrist
11. Right Wrist
12. Left Hip
13. Right Hip
14. Left Knee

15. Right Knee

16. Left Ankle

17. Right Ankle

2. Pose Estimation: Image

Use yolo26n-pose_ncnn_model to predict images in the ultralytics26 folder (non-ultralytics built-in images).

Enter the code folder:

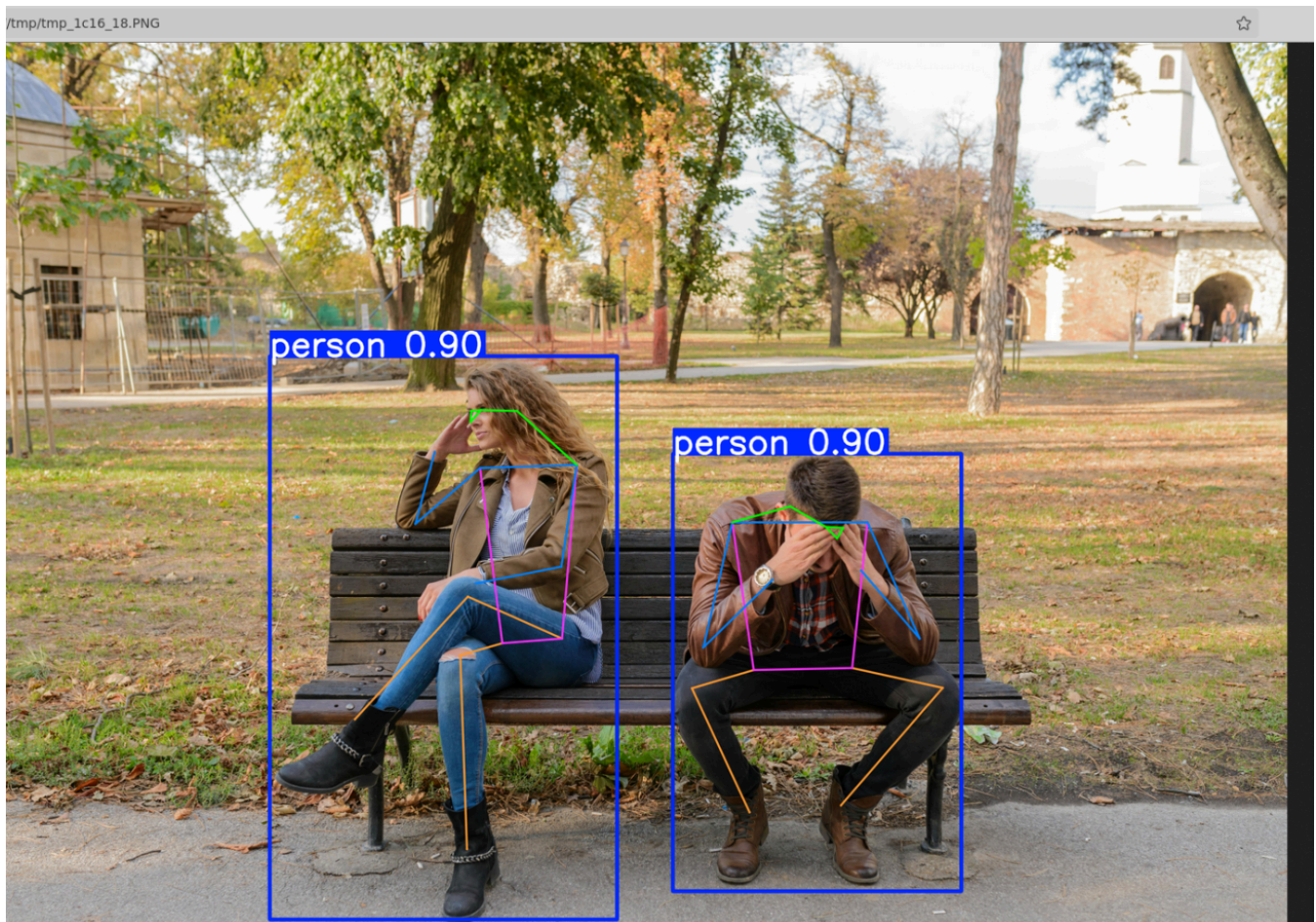
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 03.pose_image.py
```

Preview

Output image location for yolo detection: /home/sunrise/ultralytics26/output/



Example code:

```
from ultralytics import YOLO

# Load a model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-pose_ncnn_model")
```

```
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-pose.pt")

# Run batched inference on a list of images
results = model("/home/sunrise/ultralytics26/assets/people.jpg") # return a list of Results
objects

# Process results list
for result in results:
    # boxes = result.boxes # Boxes object for bounding box outputs
    # masks = result.masks # Masks object for segmentation masks outputs
    keypoints = result.keypoints # Keypoints object for pose outputs
    # probs = result.probs # Probs object for classification outputs
    # obb = result.obb # Oriented boxes object for OBB outputs
    result.show() # display to screen
    result.save(filename="/home/sunrise/ultralytics26/output/people_output.jpg") # save to
disk
```

3. Pose Estimation: Video

Use yolo26n-pose_ncnn_model to predict videos in the ultralytics26 folder (non-ultralytics built-in videos).

Enter the code folder:

```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code:

```
python3 03.pose_video.py
```

Preview

Output video location for yolo detection: /home/sunrise/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO
```

```

# Load the YOLO model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-pose_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-pose.pt")

# Open the video file
video_path = "/home/sunrise/ultralytics26/videos/people.mp4"
cap = cv2.VideoCapture(video_path)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a Videowriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/03.people_output.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your
platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

4. Pose Estimation: Real-time Detection

4.1 USB Camera

Use yolo26n-pose_ncnn_model to predict USB camera feed.

Enter the code folder:

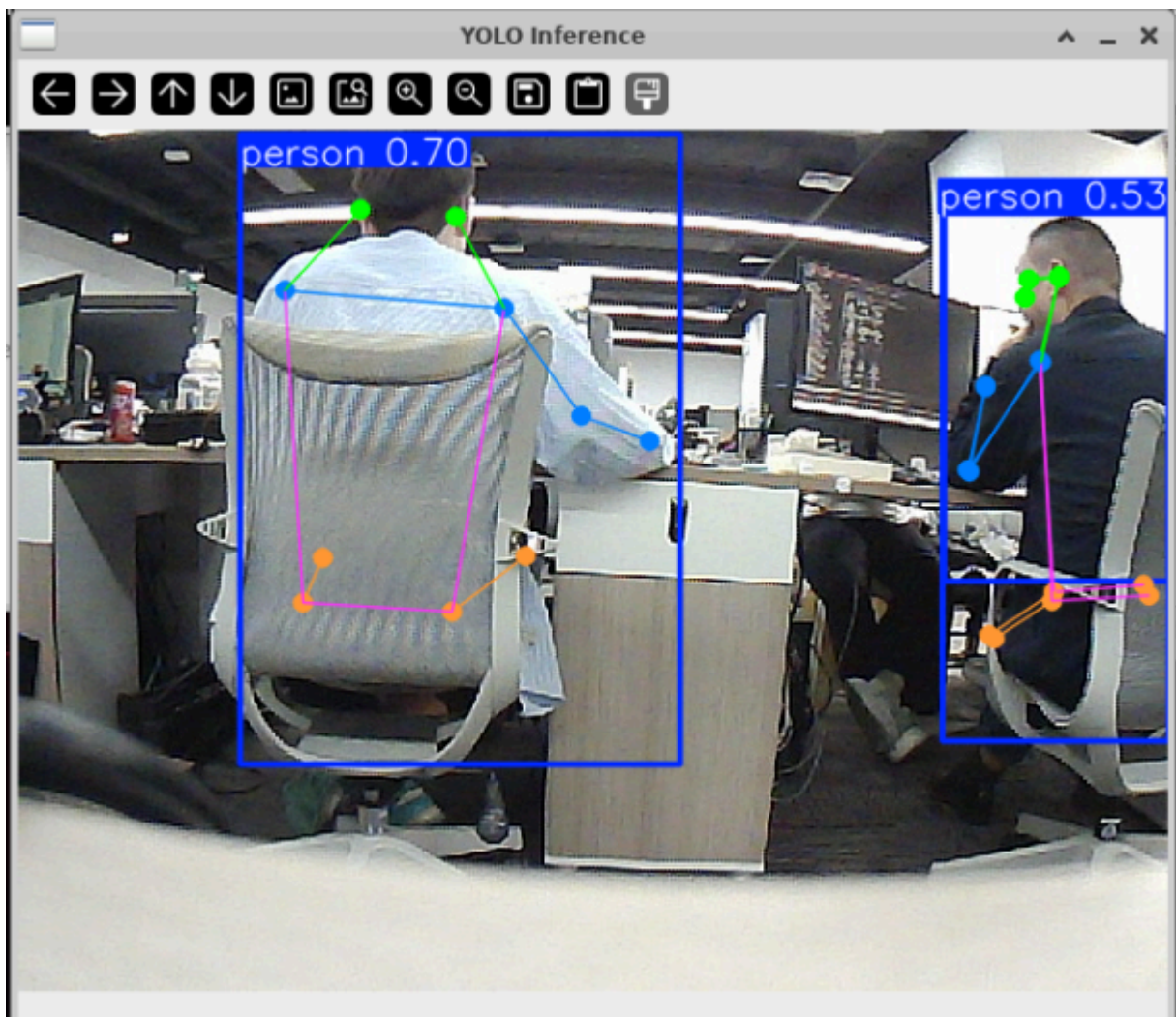
```
cd /home/sunrise/ultralytics26/yahboom_demo
```

Run the code: Click the preview window and press 'q' to terminate the program!

```
python3 03.pose_camera_usb.py
```

Preview

Output video location for yolo detection: /home/sunrise/ultralytics26/output/



Example code:

```
import cv2
from ultralytics import YOLO
```

```

# Load the YOLO model
model = YOLO("/home/sunrise/ultralytics26/yolo26n-pose_ncnn_model")
# model = YOLO("/home/sunrise/ultralytics26/yolo26n-pose.pt")

# Open the camera
cap = cv2.VideoCapture(0)
cap.set(6, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/sunrise/ultralytics26/output/03.pose_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # You can use 'XVID' or 'mp4v' depending on your
platform
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)

        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()

```

References

[Pose Estimation - Ultralytics YOLO Docs](#)